# Project Title

## Automated Network Request Management in ServiceNow

**Name** : Kavyasri Meghana Karunakaram
**College** : Aditya Institute of Technology and Management
**Email** :23a51a0523@adityatekkali.edu.in

## Project Report

| | |
|---|---|
| **Date** | 28 June 2026 |
| **Team ID** | Individual |
| **Project Name** | Automated Network Request Management project in ServiceNow |
| **Maximum Marks** | |

## Project Overview

The **Automated Network Request Management in ServiceNow** project is designed to simplify and automate the process of managing network service requests within an organization. Employees can submit requests for services such as VPN access, network connectivity, IP address allocation, device configuration, and other network-related requirements through the **ServiceNow Service Catalog**.

Instead of relying on manual processing, the system automatically validates request details, routes requests for approval, creates the necessary records, assigns tasks to the appropriate IT team, and sends email notifications at every stage. Users can also track the status of their requests in real time through the ServiceNow portal.
The solution is built using key ServiceNow features, including **Service Catalog, Catalog Variables, UI Policies, Custom Tables, Flow Designer, Approval Workflows, and Email Notifications**. By automating the entire request lifecycle, the project reduces processing time, minimizes manual errors, improves transparency, ensures compliance with organizational policies, and enhances the overall efficiency of IT service management.

**Table of Contents**



## 1. Introduction

In modern organizations, a reliable and secure network is essential for ensuring smooth business operations. Employees frequently require network-related services such as VPN access, network connectivity, IP address allocation, device configuration, and troubleshooting support. Managing these requests through manual processes often results in delays, approval bottlenecks, increased administrative effort, and a higher chance of human error.

As organizations continue to grow, the volume of network service requests also increases, making manual request management inefficient and difficult to maintain. This can lead to slower response times, reduced visibility into request progress, inconsistent approvals, and challenges in meeting Service Level Agreements (SLAs).

The **Automated Network Request Management** project, developed using the **ServiceNow** platform, provides a centralized solution for automating the entire request lifecycle. Users can submit network requests through the **Service Catalog**, while the system automatically validates request details, initiates approval workflows, creates request records, assigns tasks to the appropriate IT team, sends email notifications, and updates the request status until completion.

## Benefits of the Proposed System

- Faster processing of network service requests.
- Consistent and standardized workflow execution.
- Reduced manual effort and administrative workload.
- Improved visibility into request progress.
- Better compliance with organizational policies and SLAs.

- Accurate tracking and monitoring of network requests.

- Enhanced user satisfaction through efficient service delivery.

By leveraging the low-code capabilities of **ServiceNow**, this project demonstrates how IT Service Management (ITSM) can automate routine network operations, improve operational efficiency, and provide a scalable solution that can adapt to future business requirements.

## 2. Project Objectives :

The main objective of this project is to develop an **Automated Network Request Management System** using the ServiceNow platform to simplify and automate the handling of network service requests. The system provides a centralized platform for users to submit requests while automating approvals, notifications, record creation, and request tracking, thereby reducing manual effort and improving operational efficiency.

The specific objectives of the project are:

- Develop a centralized **Service Catalog** for submitting network service requests.
- Design dynamic request forms using **Catalog Variables** and **UI Policies**.
- Create a **Custom Table** to store and manage network request records.
- Automate the request workflow using **Flow Designer**.
- Configure approval workflows for managers and network teams.
- Generate automatic email notifications during different stages of the request lifecycle.
- Maintain request history and audit records for tracking and monitoring.
- Reduce request processing time and improve service efficiency.
- Develop a scalable solution that can support additional network services in the future.

## 3. Key Features

The proposed Automated Network Request Management System includes the following key features:

- User-friendly Service Catalog for submitting network service requests.
- Dynamic forms with intelligent field visibility based on the selected request type.
- Catalog Variables to capture all necessary request information.
- Variable Sets for reusing common user details across multiple catalog items.
- Automated approval workflows using Flow Designer.
- Custom Network Request table for storing and tracking requests.
- Automatic creation of backend request records upon submission.

## 4. Project Milestones

The project implementation follows five major milestones.

### Milestone 1: Catalog Creation

**Objective**
Establish a structured Service Catalog for network request management.

Activities:

- Identify different types of network requests.
- Create Service Catalog.
- Create Catalog Categories.
- Create Catalog Items.
- Provide meaningful names and descriptions.
- Organize request taxonomy according to ITSM standards.

### Milestone 2: Form Setup

**Objective**

Design dynamic forms to collect all necessary request information.

Activities:

- Create Catalog Variables.
- Configure Variable Sets.
- Implement Catalog UI Policies.
- Configure mandatory fields.
- Apply field validation.
- Enable dynamic show/hide functionality.

### Milestone 3: Approval Integration

**Objective**

Automate approval workflows before request fulfillment.

Activities:

- Configure approval hierarchy.
- Create Flow Designer approval actions.
- Implement manager approvals.
- Enable email notifications.

- Maintain approval audit logs.

### Milestone 4: Testing

**Objective**

Validate the complete automation workflow.

Activities:

- Unit testing.
- Catalog testing.
- Flow testing.
- Approval testing.
- Notification testing.
- End-to-end request execution.
- User Acceptance Testing (UAT).

### Milestone 5: Deployment

**Objective**

Deploy the application for organizational use.

Activities:

- Validate configurations.
- Publish catalog items.
- Monitor workflow execution.
- Train end users.
- Monitor post-deployment performance.

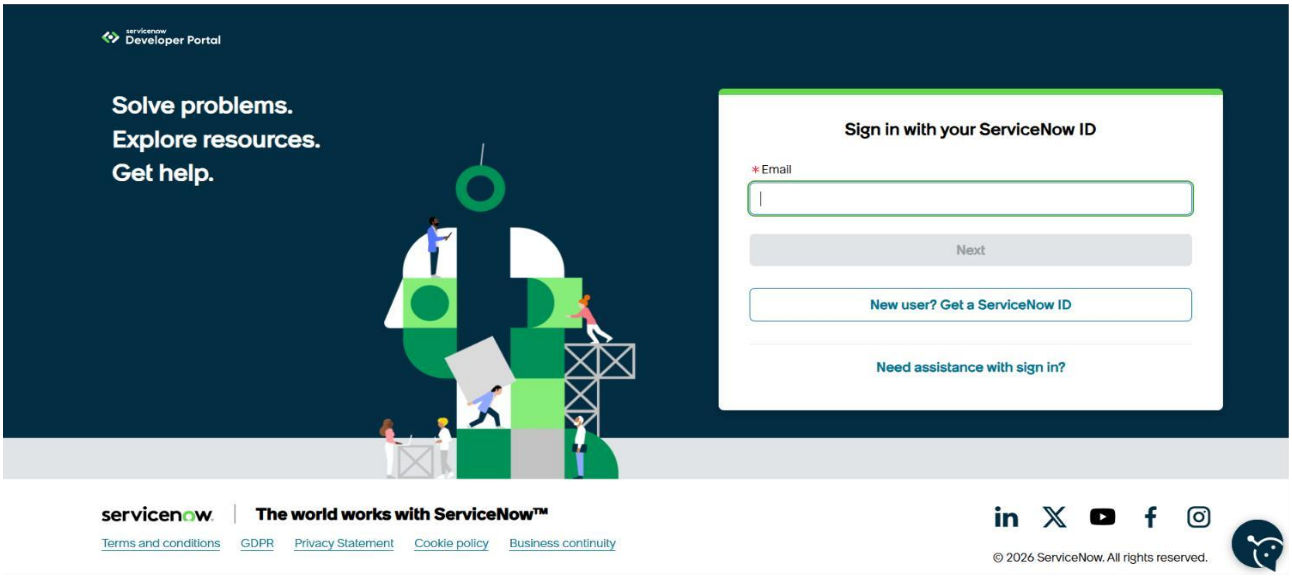
## 5. ServiceNow Developer Setup

Before developing the application, a ServiceNow Personal Developer Instance (PDI) must be created. The PDI provides a fully functional ServiceNow environment where applications can be developed, tested, and deployed without affecting production systems.

### 5.1 Creating a ServiceNow Developer Account :

To begin development, perform the following steps:

1. Visit the ServiceNow Developer Portal.
2. Click **Sign Up** to create a free developer account.
3. Enter the required information:
    - First Name
    - Last Name
    - Email Address
    - Password
4. Verify the email address using the activation link sent to your registered email.
5. Log in to the ServiceNow Developer Portal.
6. Click **Start Building**.
7. Request a **Personal Developer Instance (PDI)**.
8. Wait for the instance to be provisioned.
9. Launch the instance and log in using the generated credentials.




### 5.2 Accessing Creator Studio

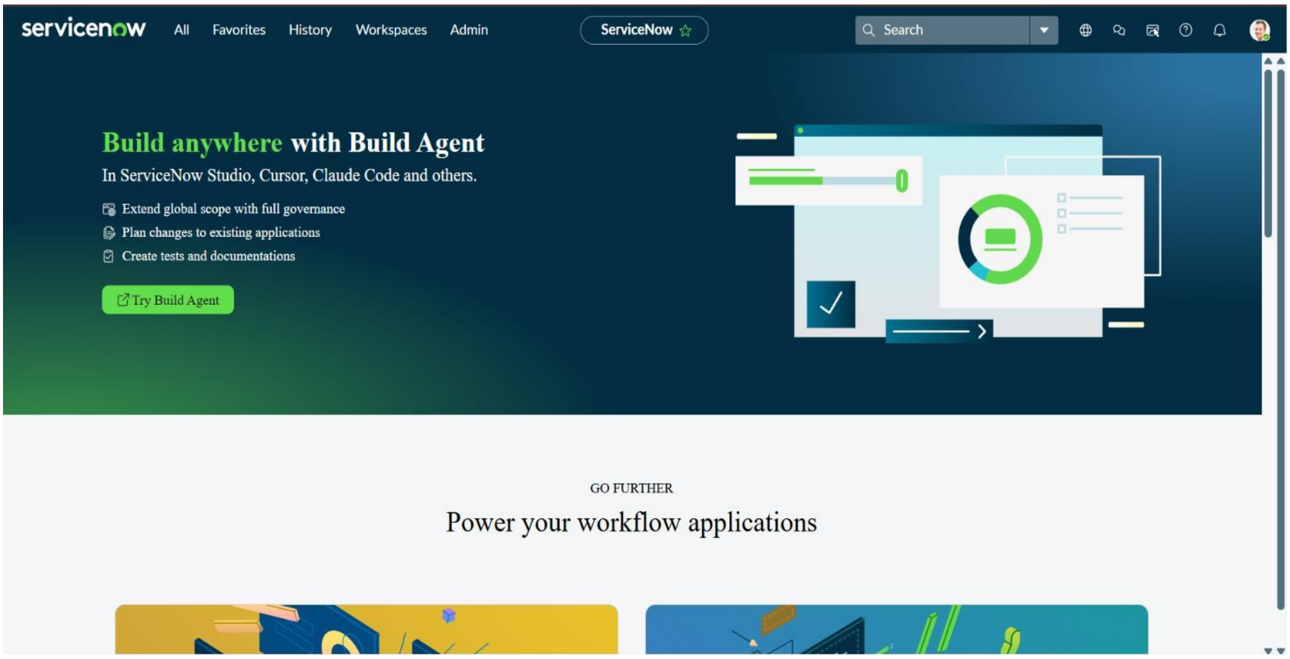
Once the Personal Developer Instance is ready, ServiceNow redirects the developer to Creator Studio.

Creator Studio provides a no-code and low-code environment for building applications. It simplifies application development by allowing developers to create forms, workflows, tables, automation, and integrations through an intuitive graphical interface.

Creator Studio offers features such as:

- Application Creation

- Data Modeling
- Flow Automation
- Form Builder
- Catalog Item Development
- Workflow Configuration

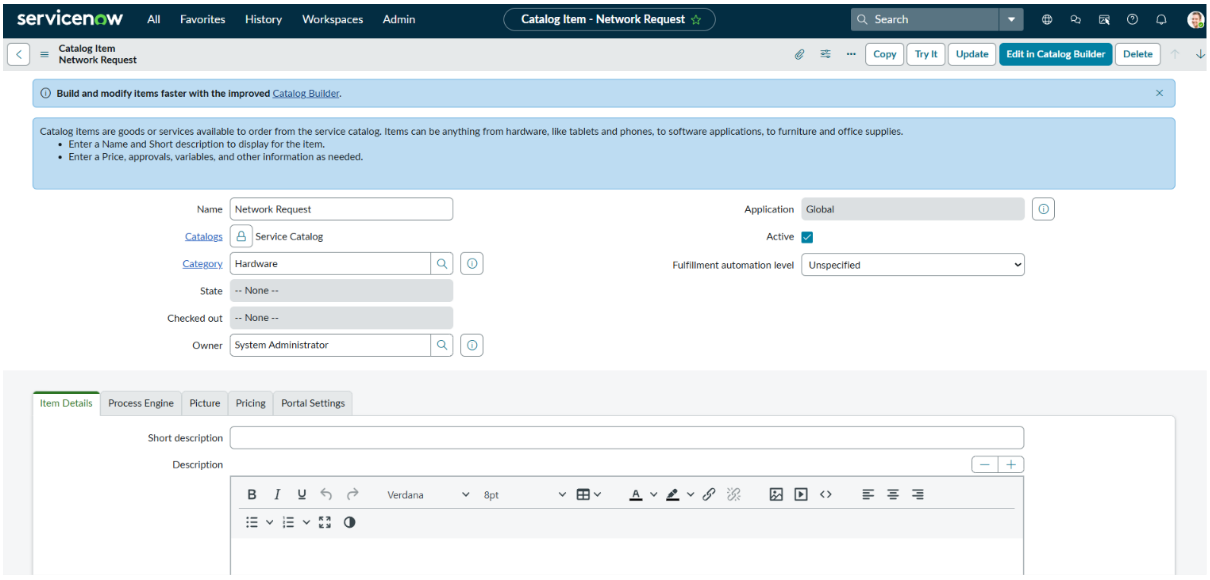



## 6.1 Service Catalog Creation

The **Service Catalog** acts as a centralized self-service portal that enables users to submit network service requests easily. In this project, a catalog item named **Network Request** was created to provide users with a single interface for requesting different network-related services such as VPN access, IP allocation, and device configuration.

## Steps to Create the Service Catalog Item

1. Open the **Application Navigator** in the ServiceNow platform.
2. Search for **Service Catalog**.
3. Navigate to the following path:
   **Service Catalog → Maintain Items**
4. Click the **New** button to create a new catalog item.
5. Enter the required details:
   - **Catalog Item Name:** Network Request
   - **Catalog:** Service Catalog
   - **Category:** Network
   - **Short Description:** Automated Network Request Management
6. Click **Save** to create and store the catalog item successfully.



## Purpose of the Catalog

The **Service Catalog** serves as the primary interface through which users can submit network-related service requests. It offers a centralized and user-friendly platform that standardizes the request submission process across the organization.

Using the **Network Request** catalog item, users can raise requests for services such as:

- New Network Connection
- Network Relocation
- Device Registration
- Network Configuration
- Connectivity Support
- Device Information Submission

By providing a single point of access for all network requests, the Service Catalog eliminates the need for manual email or phone-based requests, ensuring consistency, better tracking, and a more organized request management process.

---

## 6.2 Catalog Variables Configuration

**Catalog Variables** are used to collect the necessary information from users when they submit a network request. These variables appear as input fields on the Service Catalog form and capture the details required for processing and automating the request workflow.

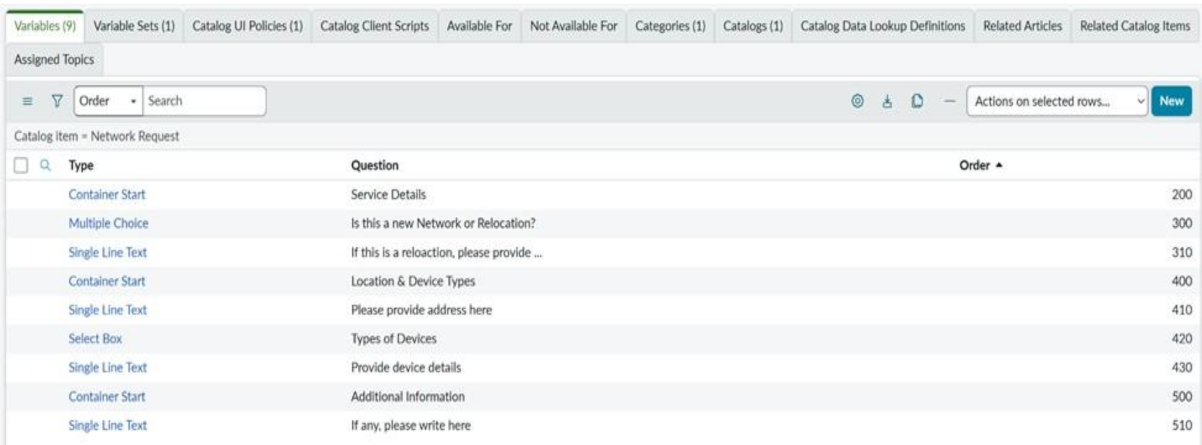
The information entered through these variables is later utilized by Flow Designer, approval processes, and other automation components within the ServiceNow platform.

## Steps to Create Catalog Variables

Follow the steps below to configure catalog variables:

1. Open the **Network Request** catalog item.
2. Navigate to the **Variables** related list.
3. Click the **New** button to add a new variable.
4. Configure the required properties, including:
   ○ Variable Type
   ○ Question
   ○ Name
   ○ Order
   ○ Tooltip
   ○ Help Text
   ○ Mandatory Status
5. Click **Save** to create the variable.

Repeat the above procedure until all required catalog variables have been added to the Network Request catalog item.




## Variable Configuration

The **Network Request** catalog item consists of different types of variables to collect all the information required for processing a request. Each variable is selected based on the type of data that needs to be captured from the user.

| Variable Name | Variable Type |
|---|---|
| Service Details | Container Start |
| Is this a New Network or Relocation? | Multiple Choice |
| If this is a relocation, please provide your previous location | Single Line Text |

| | |
|---|---|
| Location & Device Types | Container Start |
| Please provide the address here | Single Line Text |
| Types of Devices | Select Box |
| Provide Device Details | Single Line Text |
| Additional Information | Container Start |
| If any additional information, please provide here | Single Line Text |

## Types of Variables Used

**1. Reference Variable**

A **Reference Variable** is used to retrieve data from another ServiceNow table by referencing an existing record.

**Example:**

**Opened On Behalf Of → User Table**

This enables the system to automatically access user-related information, reducing manual data entry.

**2. Choice Variable**

A **Choice Variable** allows users to select one option from a predefined list of values.

**Example:**

**Network Request Type**

Available options include:

- New Connection
- Relocation
- Maintenance
- Configuration Change

**3. Single Line Text**
This variable type is used to capture short text values entered by the user.

**Examples:**

- Email Address
- Username
- Address
- Phone Number

**4. Multi-Line Text**

A **Multi-Line Text** variable is used whenever detailed information or explanations are required.

**Example: Device**

**Details Purpose:**

Allows users to provide a complete description of the device, issue, or additional technical information.

**5. Attachment Variable**

The **Attachment Variable** enables users to upload supporting documents along with their request.

Some examples include:

- Identity Proof
- Device Purchase Invoice
- Approval Documents
- Other supporting files

## Auto-Populated Variables

To improve efficiency, several fields are automatically populated using the **Dot Walking** feature. Once the user selects the **Opened On Behalf Of** field, the system automatically retrieves related information from the User table.

The automatically populated fields include:

- Email Address
- Username
- Phone Number

This automation minimizes manual input, improves accuracy, and reduces the possibility of data entry errors.

### Advantages of Catalog Variables

The use of Catalog Variables provides several benefits, including:

- Standardized collection of user information
- Improved accuracy of submitted data
- Simplified workflow automation
- Better reporting and record management
- Reduced manual verification and validation

## 6.3 Variable Set Configuration

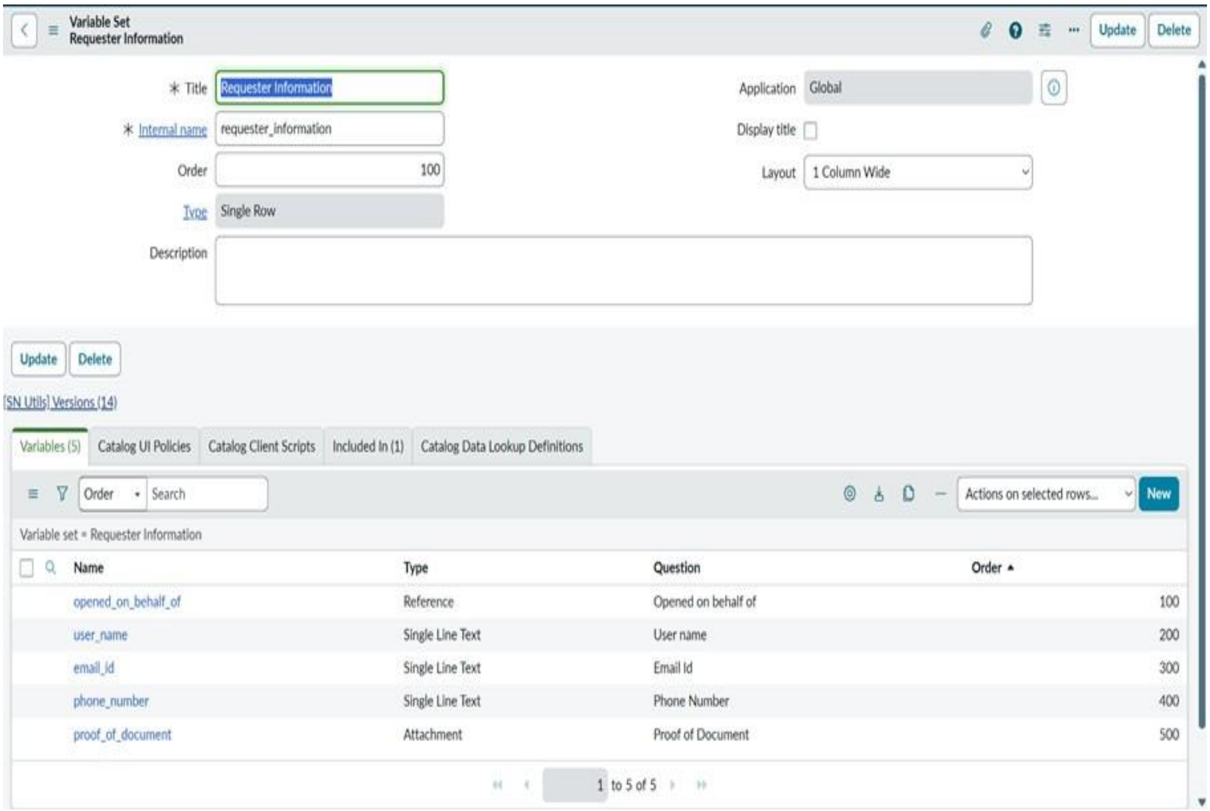
A **Variable Set** is a reusable collection of catalog variables that can be attached to multiple catalog items. Instead of creating the same variables repeatedly, developers can create them once and reuse them wherever necessary, improving consistency and reducing development time.

Some commonly reused variables include:

- Opened On Behalf Of
- Email Address
- Username
- Phone Number
- Document Upload

### Steps to Create a Variable Set

1. Open the **Application Navigator** and navigate to:
**Service Catalog → Variable Sets** 2. Click **New** to
create a new Variable Set.
3. Enter the required information:
    - Name
    - Description
4. Add all the required variables to the Variable Set.
5. Save the Variable Set.
6. Associate the created Variable Set with the **Network Request** catalog item for reuse.




### Advantages of Variable Sets

Using **Variable Sets** in the ServiceNow Service Catalog offers several benefits during application development and maintenance:

- Eliminates repetitive configuration by reusing common variables.
- Simplifies maintenance since updates are made in one place.
- Provides a consistent experience across multiple catalog items.
- Enables the same set of variables to be shared among different service requests.
- Reduces development time and improves implementation efficiency.

## 6.4 Catalog UI Policy Configuration

**Catalog UI Policies** are used to make catalog forms more interactive by controlling the visibility, mandatory status, and read-only behavior of variables based on user selections. These policies help create dynamic forms that display only the relevant fields required for a particular request.

In this project, Catalog UI Policies are configured to show or hide specific variables according to the network service selected by the user. This approach simplifies the form, reduces unnecessary data entry, and improves the overall user experience.

## Objective

The implementation of Catalog UI Policies is intended to achieve the following objectives:

- Improve the usability and appearance of the request form.
- Minimize user confusion by displaying only relevant fields.
- Dynamically show or hide variables based on user selections.
- Make fields mandatory only when they are applicable.
- Prevent incomplete or incorrect request submissions.

## Example Scenario

The UI Policies are configured to respond dynamically to user input.

For example:

- If the user selects **Device Type = Others**, an additional field named **"Please Specify Device"** is displayed.
- If the **Request Type** is selected as **Relocation**, the **Relocation Address** field becomes visible, allowing the user to enter the required location details.

This dynamic behavior ensures that users are presented only with fields relevant to their selected request.

## Steps to Configure a Catalog UI Policy

1. Open the **Application Navigator** and navigate to:
   **Service Catalog → Catalog Definitions → Maintain Items**
2. Open the **Network Request** catalog item.
3. Scroll to the **Catalog UI Policies** related list.
4. Click **New** to create a Catalog UI Policy.
5. Enter the required details:
   - **Applies To:** Catalog Item
   - **Catalog Item:** Network Request
   - **Description:** Dynamic Field Visibility
6. Configure the policy condition.
   **Example Condition:**
   - **Device Type = Others**
7. Save the Catalog UI Policy.
8. Create a **UI Policy Action**.
9. Select the variable **Please Specify Device**.
10. Set the **Visible** property to **True**.
11. Update and save the UI Policy.

12. Open the Catalog Item and verify that the fields are displayed dynamically based on the selected conditions.


### Benefits of UI Policies

Implementing **Catalog UI Policies** provides several advantages in improving the overall functionality and usability of the Service Catalog:

- Creates dynamic and interactive request forms.
- Enhances the overall user experience.
- Maintains a clean and organized interface by displaying only relevant fields.
- Minimizes validation errors during form submission.
- Improves the quality and accuracy of collected information.

## 6.5 Creation of Custom Table

A **Custom Table** was created in ServiceNow to maintain all network request records in a dedicated database. Rather than relying solely on catalog request records, this table stores complete request information, making it easier for administrators to manage, monitor, and generate reports.

Having a separate table also supports workflow automation, simplifies record management, and provides greater flexibility for future system enhancements.

### Table Details

**Table Name:** Network Database

**Purpose:**
To store and manage all submitted network requests in a centralized location for tracking, reporting, and administrative purposes.

### Steps to Create the Custom Table

1. Open the **Application Navigator**.
2. Navigate to:

**System Definition → Tables**
3. Click **New** to create a table.
4. Enter the required details:
   - **Label**
   - **Table Name**
   - **Application**
5. Enable the **Auto Generate Schema** option.
6. Click **Submit** to create the custom table.

### Advantages of the Custom Table

The custom table offers several benefits, including:

- Centralized storage for all network request records.
- Simplified report generation and data analysis.
- Improved auditing and record tracking.
- Easy integration with future enhancements.
- Independent management of request-related data.

## 6.6 Field Creation

Once the custom table was created, the required fields were added to store the information collected through the Service Catalog. Each field in the table corresponds to a catalog variable, ensuring that all request details are stored systematically.

These fields act as the database structure used by workflows, reports, and administrative processes.

### Steps to Create Table Fields

1. Open the **Network Database** custom table.
2. Navigate to the **Columns** tab.
3. Click **New** to add a new field.
4. Select the appropriate **Field Type**.
5. Enter the **Column Label**.
6. Specify the **Column Name**.
7. Save the field.

Repeat these steps until all required fields have been added to the table.

### Fields Created

| Field Name | Data Type |
| --- | --- |
| Request Number | String |
| Requested For | Reference |
| Work Status | Choice |
| Device Type | Choice |
| Device Details | String |
| Customer Address | String |
| Assignment Group | Reference |
| Date of Enquiry | Date |
| Phone Number | String |
| Email ID | String |
| Approval Status | Choice |

## 6.7 Flow Designer Implementation

**Flow Designer** is the primary automation component used in this project to manage the entire network request process. It automatically executes a sequence of predefined actions whenever a user submits a request through the **Service Catalog**. By automating tasks such as record creation, approval routing, email notifications, and status updates, the flow significantly reduces manual effort and improves the efficiency of request processing.

The automated workflow ensures that every request follows a consistent process from submission to completion while maintaining transparency and reducing processing time.

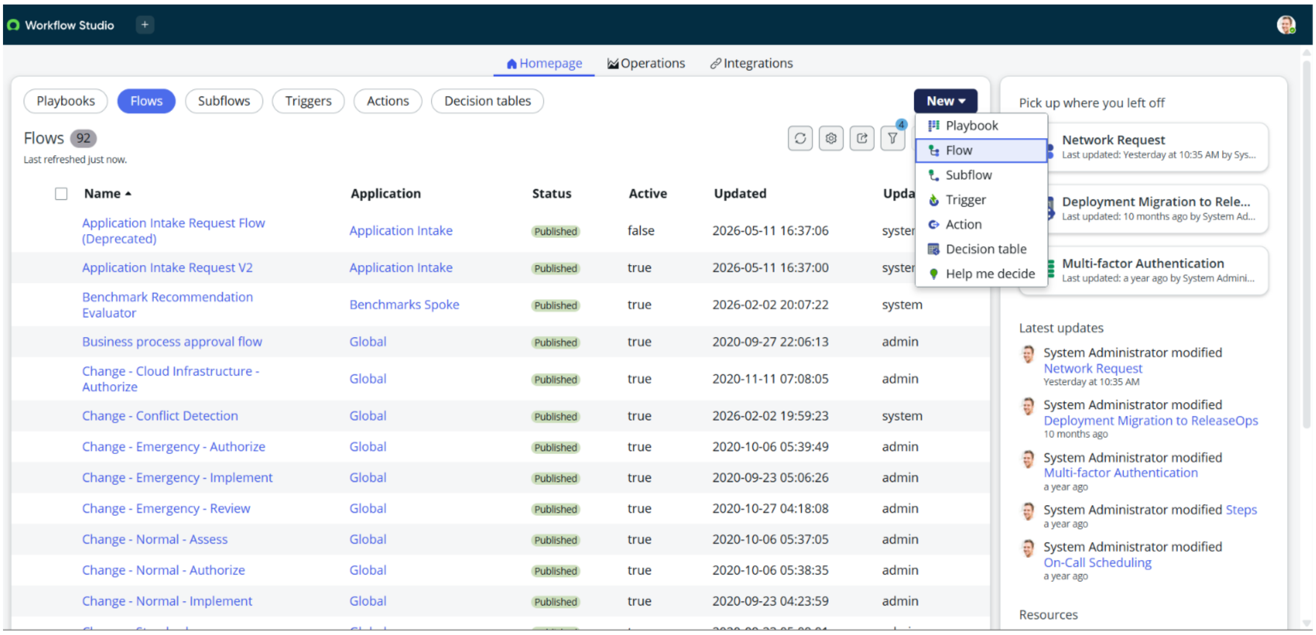
---

## 6.7.1 Flow Creation

The initial step in implementing automation is to create a new flow that controls the complete lifecycle of a network request. This flow serves as the central workflow responsible for coordinating all automated activities, including approvals, record updates, notifications, and other workflow actions.

### Steps to Create the Flow

1. Open the **Application Navigator** and search for **Flow Designer**.
2. Launch **Flow Designer**.
3. Click **New** and choose **Flow**.
4. Enter the required details:
   - **Flow Name:** Network Request Management
   - **Description:** Automates the processing of network service requests.
5. Click **Build Flow** to create the flow and begin configuring its automation logic.




## 6.7.2 Configuring the Trigger

The **Trigger** specifies the event that initiates the execution of the flow. In this project, the flow is configured to start automatically whenever a user submits the **Network Request** catalog item through the Service Catalog.

This ensures that the automation process begins immediately after a request is created, eliminating the need for manual intervention.

### Steps to Configure the Trigger

1. Click the **+ (Add)** icon in the Flow Designer.
2. Select **Trigger**.
3. Choose **Service Catalog** as the trigger type.
4. Select **Requested Item Created**.

5. Specify the **Network Request** catalog item.
6. Save the trigger configuration.

## 6.7.3 Get Catalog Variables

After the flow is triggered, the next step is to retrieve the information entered by the requester. This is accomplished using the **Get Catalog Variables** action, which collects all the required input values from the submitted catalog item.

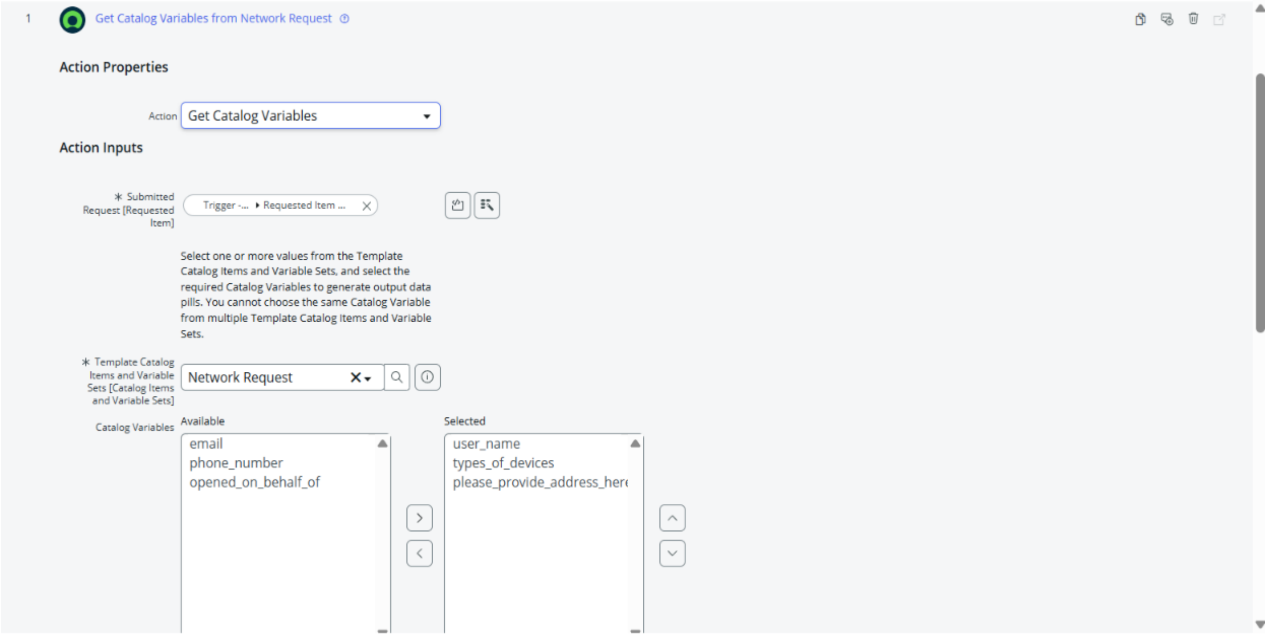
The retrieved data is then used throughout the workflow for record creation, approval processing, notifications, and other automated actions.

### Steps to Configure Get Catalog Variables

1. Click **Add Action** within the flow.
2. Search for **Get Catalog Variables**.
3. Select the **Get Catalog Variables** action.
4. Configure the **Requested Item** as the input parameter.
5. Choose all the required catalog variables.
6. Save the action configuration.

The action retrieves important request details, including:

- Requested For
- Email ID
- Phone Number
- Device Type
- Device Details
- Address
- Request Type




## 6.7.4 Create Record

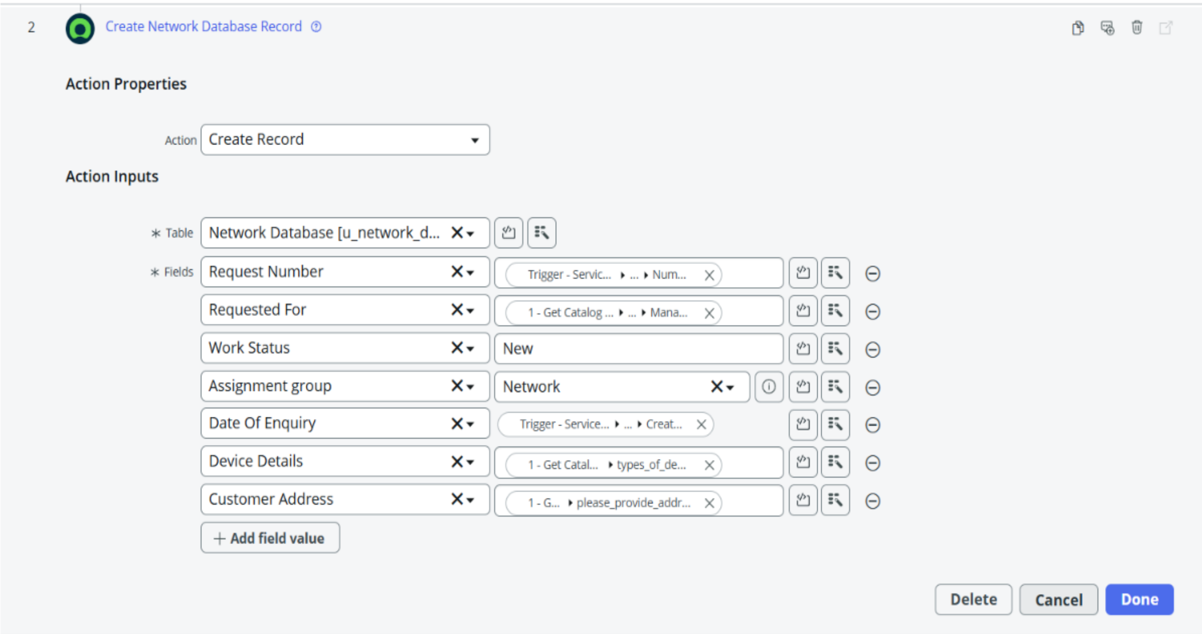
Once the catalog variables have been retrieved, the next step is to create a new record in the **Network Database** custom table. The information collected from the Service Catalog is mapped to the corresponding fields in the table, ensuring that all request details are stored in a structured manner.

This record serves as the central source for tracking, managing, and processing network service requests throughout their lifecycle.

### Steps to Configure the Create Record Action

1. Click **Add Action** in the Flow Designer.
2. Select the **Create Record** action.
3. Choose the **Network Database** custom table.
4. Map each catalog variable to its corresponding field in the table.
5. Save the action configuration.

After the record is created successfully, all relevant request information is stored in the custom table, enabling administrators to monitor request status, generate reports, and efficiently manage network service requests.




## 6.7.5 Approval Process

To ensure that network requests are processed only after proper authorization, an **Approval Process** is incorporated into the workflow. Once a request record is created, it is automatically forwarded to the designated approver, such as a **Manager** or **Network Administrator**, for review and decision.

This approval mechanism helps maintain organizational policies, improves accountability, and ensures that only valid requests proceed to the next stage of processing.

### Steps to Configure the Approval Process

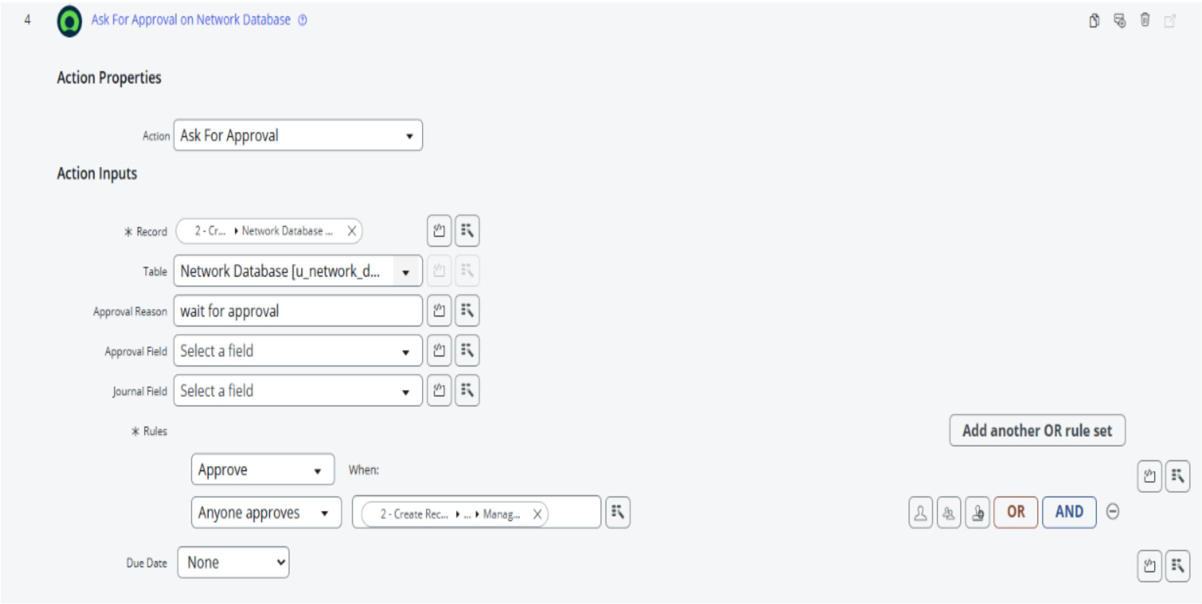
1. Click **Add Action** in the Flow Designer.
2. Select the **Ask for Approval** action.
3. Choose the newly created **Network Database** record as the target record.
4. Configure the required approval rule.
5. Specify the appropriate approver (Manager or Network Administrator).
6. Enter the approval reason and save the configuration.

### Approval Outcomes

After the approver reviews the request, one of the following outcomes is recorded:

- **Approved**
- **Rejected**
- **Pending**

ServiceNow automatically records the approval history, including the approver's decision and status, providing a complete audit trail for every network request.




## 6.7.6 Flow Logic

The workflow uses **conditional logic** to determine the next set of actions based on the approval decision. After the approver reviews the request, the flow automatically follows the appropriate execution path, ensuring that each request is processed according to its approval status.

### Approved Request

If the request receives approval, the flow performs the following actions:

- Sends an approval notification email to the requester.
- Updates the request status to **Approved** in the Network Database.
- Continues with the remaining request fulfillment process.

### Rejected Request

If the request is rejected, the workflow performs these actions:

- Sends a rejection notification email to the requester.
- Changes the request status to **Rejected**.
- Stops any further processing of the request.

By implementing this decision-based logic, the workflow ensures that only authorized and approved network requests proceed to the execution stage, while rejected requests are closed with appropriate notifications and status updates.

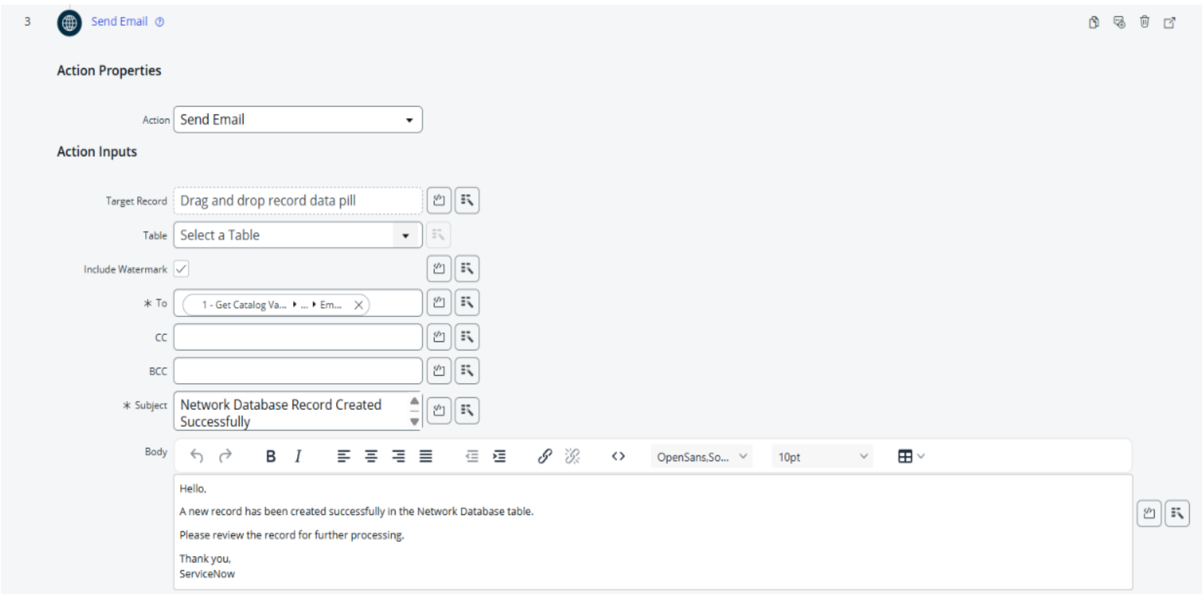

## 6.7.7 Email Notifications

To keep users updated throughout the request lifecycle, the system generates **automatic email notifications** at key stages of the workflow. These notifications provide timely updates about the current status of the network request, improving communication between users and administrators.

Email notifications are triggered during the following stages:

- Request Submitted
- Approval Pending
- Request Approved
- Request Rejected
- Request Completed

Each notification includes essential request information, such as the request details and its current status. This ensures that users remain informed throughout the entire process, enhances transparency, and reduces the need for manual follow-up regarding request progress.



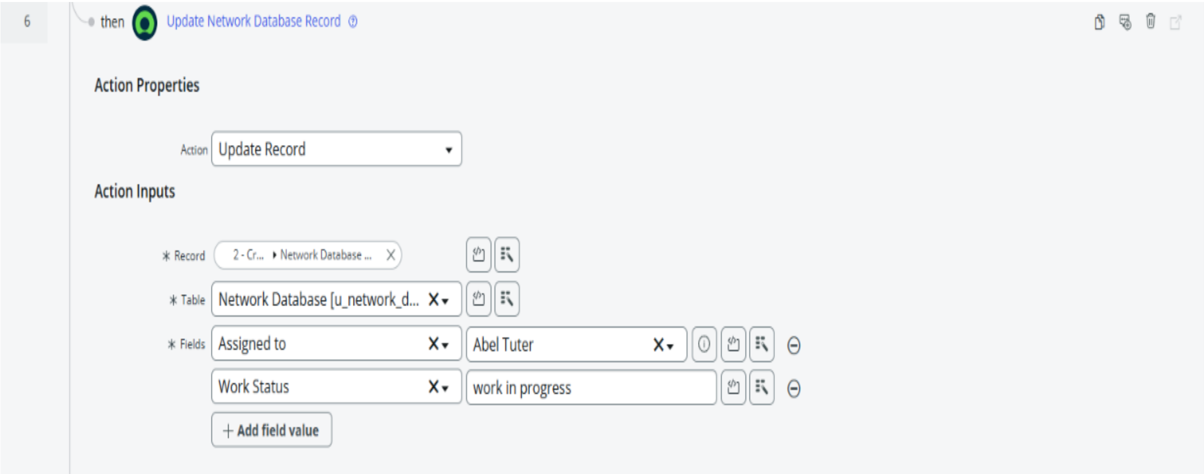
## 6.7.8 Update Record

After the approval process is completed, the corresponding entry in the **Network Database** custom table is updated automatically. The workflow modifies the record with the latest information so that the request status remains accurate throughout its lifecycle.

The system updates the following details:

- Approval Status
- Work Status
- Completion Date
- Assigned Team

By automatically updating these fields, the database always reflects the most recent progress of each network request. This ensures accurate record management, simplifies request tracking, and provides administrators with up-to-date information for monitoring and reporting purposes.



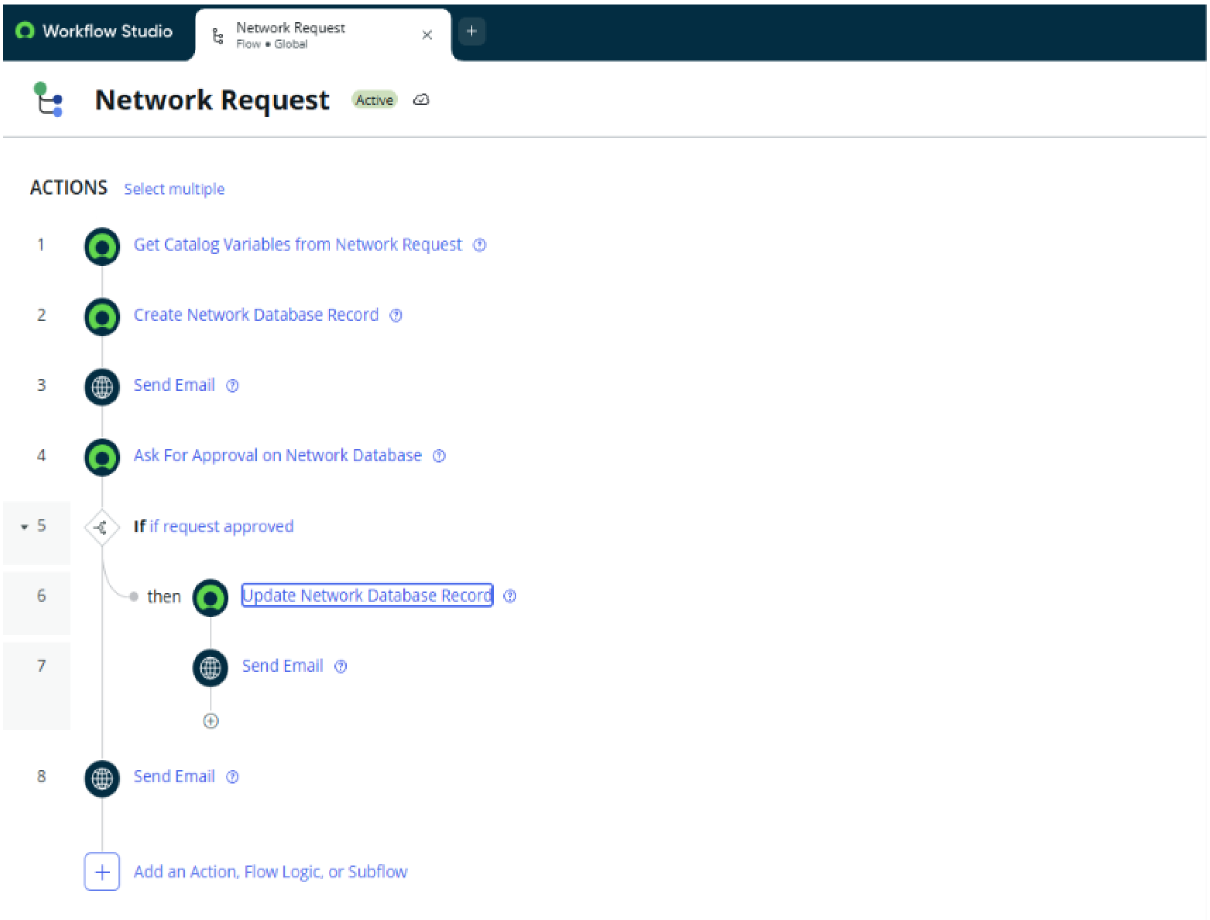

## 6.9 Workflow Summary

The following sequence outlines the complete workflow implemented in the **Automated Network Request Management** system. It demonstrates how a network request is processed from submission to completion using ServiceNow automation.

1. The user submits a **Network Request** through the **Service Catalog**.
2. The configured **Flow Designer** is automatically triggered upon request submission.
3. The flow retrieves all required **Catalog Variables** entered by the user.
4. A new record is generated in the **Network Database** custom table to store the request details.
5. An approval request is automatically forwarded to the designated approver, such as a Manager or Network Administrator.
6. The approver reviews the submitted request and records the approval decision.
7. Based on the approval outcome:
    - **Approved:** The request continues to the fulfillment stage, and an approval notification is sent to the requester.

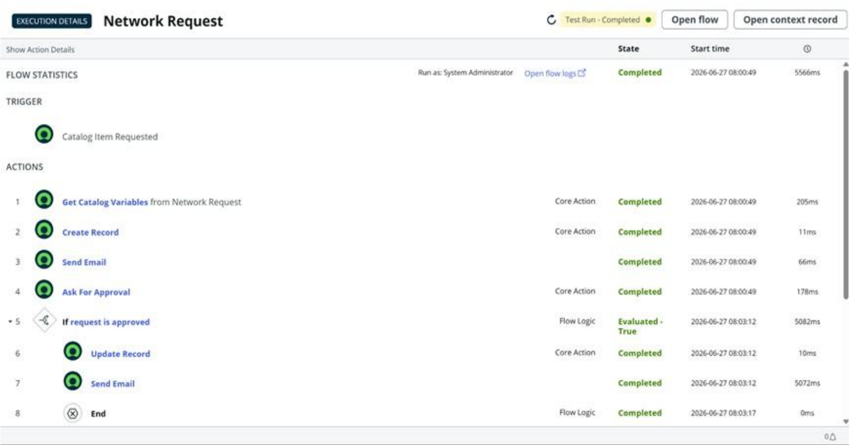
○ **Rejected:** The request is marked as rejected, the workflow stops further processing, and a rejection notification is sent to the requester.

8. Finally, the corresponding record in the **Network Database** table is updated with the latest request status and other relevant details..

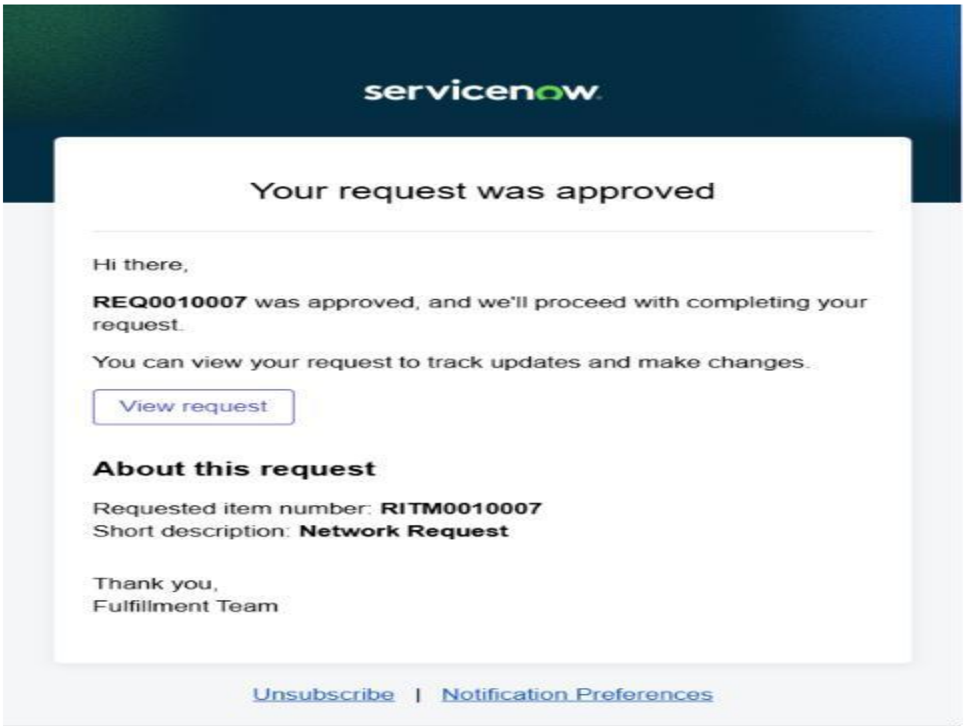



# 7. Screenshots of Output

## 1. Flow Execution Test



**2. Email Notification**




# 8. Conclusion

The **Automated Network Request Management** project successfully demonstrates how the **ServiceNow** platform can simplify and automate the handling of network service requests. By utilizing features such as the **Service Catalog, Catalog Variables, UI Policies, Custom Tables, Flow Designer, Approval Workflows, and Email Notifications**, the system provides a structured and efficient process for managing requests from submission to completion.

The automated workflow minimizes manual effort, accelerates request processing, and ensures that every request follows a consistent approval and fulfillment process. Features such as automated notifications, real-time status updates, and centralized record management improve communication, enhance transparency, and help organizations meet their Service Level Agreements (SLAs).

Overall, this project offers a reliable, scalable, and user-friendly solution for managing network requests. It improves operational efficiency, reduces processing errors, and provides a strong foundation for future enhancements, including the integration of additional network services and advanced automation capabilities.